

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES



Manual Tecnico

ELI-NOSQL

Christopher Miguel Angel Ramos Ascencio
202200057

Requisitos del sistema

Componente	Requisito mínimo	Notas importantes
Sistema Operativo	Windows 10 / Linux (Ubuntu 20.04+) / macOS Catalina	Compatible con VS Code y Node.js
Editor	Visual Studio Code (última versión estable)	Extensiones: ESLint, Prettier, React Developer Tools
Node.js	v16.x o superior	Necesario para ejecutar backend en Jison y frontend
npm / yarn	npm v8.x o Yarn v1.22+	Para instalar dependencias
Jison	v0.4.18 o superior	Generador de parser para backend
React	v18.x	Framework frontend
Browser	Chrome / Edge / Firefox (última versión)	Para pruebas del frontend
RAM	8 GB mínimo	Recomendado 16 GB para proyectos grandes
CPU	Procesador 64 bits, 2 núcleos	Mejor rendimiento con 4 núcleos
Disco	2 GB libres para dependencias y caché	Espacio adicional para node_modules
Git	v2.30+	Control de versiones y despliegue

Como configurar el Entorno

Etapa	Comando desde terminal/gitbash	Resultado esperado
Generar parser.js	./generador.sh	Se crea el archivo parser.js
Ejecutar pruebas (En la terminal)	cd Backend → cd Test → node Test.js	Se ejecutan pruebas del backend
Inicializar Backend	cd Backend → node index.js	Servidor activo en http://localhost:4000
Inicializar Frontend	cd Frontend → npm install → npm run dev	Frontend activo en http://localhost:5173/

IMPORTANTE

Cada comando correrlo en una terminal diferente

Configuración JISON

parser.jison:

Es el archivo fuente de la gramática. Contiene la definición de los tokens (Lexer) y las reglas sintácticas (Parser). Aquí es donde se define cómo se escribe un if, un for o una función en GoScript. Es el "manual de instrucciones" para construir el AST.

```
Backend > Lenguaje > Parser.json
```

```
1  %{  
2  |    const { agregarError } = require('../Util/Errores');  
3  |  }%  
4  
5
```

Reservadas:

```
Parserjison M X
Backend > Lenguaje > Parserjison
14  "/*"["^\n"]*          /* comentario de una línea */
15  "/*"["\s\S"]+?*/      /* comentario multilinea */
16
17  "true"                return 'TRUE';
18  "false"               return 'FALSE';
19  "nil"                 return 'NIL';
20
21  "int"                 return 'T_INT';
22  "float64"             return 'T_FLOAT64';
23  "string"              return 'T_STRING';
24  "bool"                return 'T_BOOL';
25  "rune"                return 'T_RUNE';
26  "var"                 return 'VAR';
27  "const"               return 'CONST';
28  "func"                return 'FUNC';
29  "return"              return 'RETURN';
30  "if"                  return 'IF';
31  "else"                return 'ELSE';
32  "for"                 return 'FOR';
33  "break"               return 'BREAK';
34  "continue"            return 'CONTINUE';
35  "switch"              return 'SWITCH';
36  "case"                return 'CASE';
37  "default"             return 'DEFAULT';
38  "struct"              return 'STRUCT';
39  "range"               return 'RANGE';
40  "main"                return 'MAIN';
41  "fmt"                 return 'FMT';
42  "Println"             return 'PRINTLN';
43  "println"             return 'PRINTLN';
44  "Print"               return 'PRINT';
45  "print"               return 'PRINT';
46  "strconv"             return 'STRCONV';
47  "Atoi"                return 'ATOI';
48  "ParseFloat"          return 'PARSEFLOAT';
49  "reflect"             return 'REFLECT';
50  "TypeOf"              return 'TYPEOF';
51  "slices"              return 'SLICES';
52  "Index"               return 'INDEX';
53  "strings"             return 'STRINGS';
54  "Join"                return 'JOIN';

Backend > Lenguaje > Parserjison
57
58  "[0-9]+","-[0-9]+([eE][+-]?[0-9]+)?" return 'LIT_FLOAT';
59  "[0-9]+" return 'LIT_INT';
60
61  "\\"["\\\\"\\nrt][\\"'\\\\"]*" return 'LIT_STRING';
62  "\\"["\\\\"\\nrt][\\"'\\\\"]\\" return 'LIT_RUNE';
63
64  "[a-zA-Z_][a-zA-Z0-9_]*" return 'ID';
```

Simbolos:

```
65
66 "++"      return 'INC';
67 "--"      return 'DEC';
68 "+"       return 'PLUS';
69 "-"       return 'MINUS';
70 "*"       return 'TIMES';
71 "/"       return 'DIVIDE';
72 "%"       return 'MOD';
73
74 ":@"      return 'DECL_ASSIGN';
75 "+="      return 'PLUS_ASSIGN';
76 "-="      return 'MINUS_ASSIGN';
77 "*="      return 'TIMES_ASSIGN';
78 "/="      return 'DIVIDE_ASSIGN';
79 "%="      return 'MOD_ASSIGN';
80 "="       return 'ASSIGN';
81
82 "=="      return 'EQ';
83 "!="      return 'NEQ';
84 "<="      return 'LTE';
85 ">="      return 'GTE';
86 "<"       return 'LT';
87 ">"       return 'GT';
88
89 "&&"      return 'AND';
90 "||"      return 'OR';
91 "!"       return 'NOT';
92
93 "("       return 'LPAREN';
94 ")"       return 'RPAREN';
95 "{"       return 'LBRACE';
96 "}"       return 'RBRACE';
97 "["       return 'LBRACKET';
98 "]"       return 'RBRACKET';
99 ","       return 'COMMA';
100 "."       return 'DOT';
101 ":"       return 'COLON';
102
103 <<EOF>>   return 'EOF';
```

Precedencia:

```
111 /* =====
112 | PRECEDENCIA – de menor a mayor
113 | ===== */
114 %right  ASSIGN DECL_ASSIGN PLUS_ASSIGN MINUS_ASSIGN TIMES_ASSIGN DIVIDE_ASSIGN MOD_ASSIGN
115 %left   OR
116 %left   AND
117 %left   EQ NEQ
118 %left   LT GT LTE GTE
119 %left   PLUS MINUS
120 %left   TIMES DIVIDE MOD
121 %right  UMINUS NOT
122 %left   INC DEC
123 %left   LBRACKET RBRACKET
124 %left   DOT
125 %left   LPAREN RPAREN
126
127 %start programa
128
```

Producciones:

(Para conocer mas consultar DOCU/Gramatica.md)

```
164 /* — func main — */
165 func_main
166 : FUNC MAIN LPAREN RPAREN bloque
167   { $$ = { tipo: 'func_main', cuerpo: $5 }; }
168 ;
169
170 /* — Structs — */
171 decl_struct
172 : STRUCT ID LBRACE nl_opt campos_struct RBRACE
173   { $$ = { tipo: 'decl_struct', nombre: $2, campos: $5 }; }
174 ;
175
176 campos_struct
177 : campos_struct campo_struct { $1.push($2); $$ = $1; }
178 | campo_struct { $$ = [$1]; }
179 ;
180
181 campo_struct
182 : tipo ID SEMICOLON nl_opt { $$ = { tipoDato: $1, nombre: $2 }; }
183 | tipo ID NL nl_opt { $$ = { tipoDato: $1, nombre: $2 }; }
184 ;
185
186 /* — Tipos — */
187 tipo
188 : T_INT { $$ = 'int'; }
189 | T_FLOAT64 { $$ = 'float64'; }
190 | T_STRING { $$ = 'string'; }
191 | T_BOOL { $$ = 'bool'; }
192 | T_RUNE { $$ = 'rune'; }
193 | LBRACKET RBRACKET tipo { $$ = '[' + $3; }
194 | ID { $$ = $1; }
195 ;
```

parser.js: Es el archivo **generado** por Jison a partir del .jison. No se edita a mano. Es el código JavaScript real que toma una cadena de texto y la convierte en un árbol de objetos (AST) siguiendo las reglas que se le definieron previamente

```
Parserjison M JS Parserjs M X
Backend > Language > JS Parserjs > Parser > <function> > parser > parse
73 /*
74 var Parser = (function(){
75 var o=function(k,v,o,l){for(o=o||[],l=k.length;l-->0;k[l]=v);return o},$V0=[2,5,7,16,21,22,35,40],$V1=[1,12],$V2=[1,14],$V3=[1,19],$V4=[1,18],$V5=[1,24],$V6=[1,23]
76 var parser = {trace: function trace () { },
77 yy: {},
78 symbols_: {"error":2,"programa":3,"declaraciones_globales":4,"EOF":5,"nl_opt":6,"NL":7,"terminador":8,"SEMICOLON":9,"declaracion_global":10,"decl_variable":11,"dec
79 terminals_: {2:"error",5:"EOF",7:"NL",9:"SEMICOLON",16:"FUNC",17:"MAIN",18:"LPAREN",19:"RPAREN",21:"STRUCT",22:"ID",23:"LBRACE",25:"RBRACE",28:"T_INT",29:"T_FLOAT6
80 productions_: [[0,[3,2],[6,2],[6,0],[8,1],[8,1],[4,2],[4,0],[10,2],[10,2],[10,1],[10,1],[10,1],[10,1],[14,5],[15,6],[24,2],[24,1],[26,4],[26,4],[27,1],[27,1]
81 performAction: function anonymous(yytext, yyleng, yylineno, yy, yystate /* action[1] */, $$/ /* vstack */, _$ /* lstack */) {
82 /* this == yyval */
83
84 var $0 = $.length - 1;
85 switch (yystate) {
86 case 1:
87   return $$[$0-1];
88   break;
89 case 2: case 3: case 13: case 14: case 58: case 59: case 87: case 91: case 110:
90   this.$ = null;
91   break;
92 case 6: case 17: case 93:
93   $$[$0-1].push($$[$0]); this.$ = $$[$0-1];
94   break;
95 case 7: case 38: case 44: case 94: case 98: case 126:
96   this.$ = [];
97   break;
98 case 8: case 9: case 49: case 50: case 51: case 52: case 53: case 54: case 57: case 103: case 104: case 105: case 106: case 107: case 108: case 109: case 144:
99   this.$ = $$[$0-1];
100   break;
101 case 10: case 11: case 12: case 27: case 37: case 45: case 46: case 47: case 48: case 85: case 86: case 88: case 99: case 100: case 101: case 102: case 125: case 1
102   this.$ = $$[$0];
103   break;
104 case 15:
105   this.$ = { tipo: 'func_main', cuerpo: $$[$0] };
106   break;
107 case 16:
108   this.$ = { tipo: 'decl_struct', nombre: $$[$0-4], campos: $$[$0-1] };
109   break;
110 case 18: case 40: case 128: case 166: case 171: case 173:
111   this.$ = $$[$0];
112   break;
113 case 19: case 20:
```

Configuración Backend y Test

index.js:

Es el punto de entrada del servidor (Backend). Aquí se configuro Express para recibir el código desde el Frontend (React), mandarlo a analizar al parser, capturar los errores y devolver los reportes y el AST al cliente.

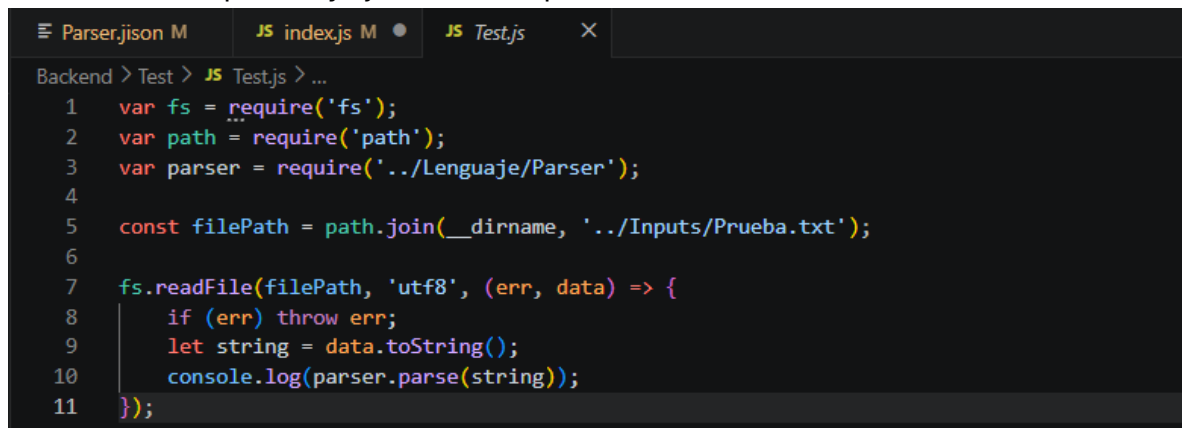
```
Backend > JS indexjs > ...
1  const express = require('express');
2  const cors = require('cors');
3  const parser = require('./Lenguaje/Parser'); // Tu parser de Jison
4  const { listaErrores, limpiarErrores } = require('./Util/Errores');
5  const { generarHTMLReporte } = require('./Util/Reportes');
6  const { listaSimbolos } = require('./Util/TablaSimbolos');
7  const interprete = require('./Lenguaje/Interprete');
8
9  // NUEVO
10 const { generarDOT } = require('./Util/GeneradorAST');
11
12 // nuevo
13 let ultimoAST = null; // Variable para persistir el árbol
14
15
16 const app = express();
17 app.use(cors()); // Permite comunicación con React
18 app.use(express.json()); // Permite leer JSON en las peticiones
19
20 // RUTA PARA ANALIZAR
21 app.post('/analizar', (req, res) => {
22   const { codigo } = req.body;
23   try {
24     const ast = parser.parse(codigo); // Ejecuta el parser
25     ultimoAST = ast;
26
27     console.log("AST guardado correctamente"); // Agrega este log para estar seguro
28     res.json({ ast, errores: listaErrores });
29   } catch (e) {
30     res.status(500).json({ error: e.message });
31   }
32 });
```

get Reportes y localhost:

```
77 // RUTA PARA LOS REPORTES
78 // Ruta para Errores
79 app.get('/reporte-errores', (req, res) => {
80   const { listaErrores } = require('./Util/Errores');
81   const { generarHTMLReporte } = require('./Util/Reportes');
82
83   const html = generarHTMLReporte('ERRORES', listaErrores);
84   res.send(html);
85 });
86
87 // Ruta para Símbolos
88 app.get('/reporte-tabla', (req, res) => {
89   const { listaSimbolos } = require('./Util/TablaSimbolos');
90   const { generarHTMLReporte } = require('./Util/Reportes');
91
92   const html = generarHTMLReporte('SIMBOLOS', listaSimbolos);
93   res.send(html);
94 });
95
96 app.listen(4000, () => console.log("🚀 Servidor Backend en http://localhost:4000"));
97
```

Test.js:

Es un archivo que usaba para hacer pruebas en consola llamando directamente la información de prueba y ejecutando el parser



```
Backend > Test > JS Test.js > ...
1  var fs = require('fs');
2  var path = require('path');
3  var parser = require('../Lenguaje/Parser');
4
5  const filePath = path.join(__dirname, '../Inputs/Prueba.txt');
6
7  fs.readFile(filePath, 'utf8', (err, data) => {
8    if (err) throw err;
9    let string = data.toString();
10   console.log(parser.parse(string));
11 });
```


Reportes

Errores.js:

Gestiona la **Tabla de Errores**. Contiene una clase Error y una lista global donde se guardan los errores léxicos, sintácticos y semánticos con su descripción, línea y columna para el reporte final.

```
Backend > Util > JS Errores.js > ...
1  class ErrorCustom {
2      constructor(tipo, descripcion, linea, columna) {
3          this.tipo = tipo;
4          this.descripcion = descripcion;
5          this.linea = linea;
6          this.columna = columna;
7      }
8  }
9
10 const listaErrores = [];
11
12 const agregarError = (tipo, desc, lin, col) => {
13     listaErrores.push(new ErrorCustom(tipo, desc, lin, col));
14 };
15
16 const limpiarErrores = () => { listaErrores.length = 0; };
17
18 module.exports = {
19     listaErrores,
20     agregarError,
21     limpiarErrores
22 };
```

TablaSimbolos.js:

Define la clase Simbolo y la lista que almacena todas las variables, funciones y constantes detectadas. Para luego ser usadas por la clase de reportes

```
Backend > Util > JS TablaSimbolos.js > ...
1  class Simbolo {
2      constructor(id, tipoSimbolo, tipoDato, ambito, linea, columna) {
3          this.id = id;
4          this.tipoSimbolo = tipoSimbolo;
5          this.tipoDato = tipoDato;
6          this.ambito = ambito;
7          this.linea = linea;
8          this.columna = columna;
9      }
10 }
11
12 const listaSimbolos = [];
13
14 module.exports = {
15     Simbolo,
16     listaSimbolos
17 };
```

Reportes.js:

Es el generador de visualización. Toma las listas de errores y de símbolos y las convierte en código **HTML** puro creando unas tablas. Es lo que permite que al dar clic en los botones de frontend de cada reporte se abra una pestaña nueva con información legible.

```
Backend > Util > JS Reportes.js > [?] generarHTMLReporte
1  const generarHTMLReporte = (tipo, datos) => {
2    const listaSimbolos = [];
3    const esError = tipo === 'ERRORES';
4    const titulo = esError ? "Reporte de Errores" : "Tabla de Símbolos";
5    const colorHeader = esError ? "#d9534f" : "#5cb85c";
6
7    let tablaEncabezado = esError
8      ? `<thead><tr><th>No.</th><th>Descripción</th><th>Línea</th><th>Columna</th><th>Tipo</th></tr></thead>`
9      : `<thead><tr><th>ID</th><th>Tipo Símbolo</th><th>Tipo Dato</th><th>Ámbito</th><th>Línea</th><th>Columna</th></tr></thead>`;
10
11    let filas = "";
12    datos.forEach((item, index) => {
13      if (esError) {
14        filas += `<tr><td>${index+1}</td><td>${item.descripcion}</td><td>${item.linea}</td><td>${item.columna}</td><td>${item.tipo}</td></tr>`;
15      } else {
16        filas += `<tr><td>${item.id} || ''</td><td>${item.tipoSímbolo} || ''</td><td>${item.tipoDato} || ''</td><td>${item.ambito} || ''</td><td>${item.lin
17      }
18    });
19
20    return `
21    <!DOCTYPE html>
22    <html>
23      <head>
24        <meta charset="UTF-8">
25        <title>${titulo}</title>
26        <style>
27          body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; padding: 20px; background-color: #f9f9f9; }
28          h2 { color: #333; }
29          table { width: 100%; border-collapse: collapse; margin-top: 20px; background-color: white; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
30          th, td { border: 1px solid #ddd; padding: 12px; text-align: left; }
31          th { background-color: ${colorHeader}; color: white; text-transform: uppercase; }
32          tr:nth-child(even) { background-color: #f2f2f2; }
33          tr:hover { background-color: #ddd; }
34        </style>
35      </head>
36      <body>
37        <h2>${titulo}</h2>
38        <table>${tablaEncabezado}<tbody>${filas}</tbody></table>
39      </body>
40    </html>`;
```

generadorAST.js:

Contiene la lógica para transformar el objeto JSON que devuelve Jison en un formato compatible con (lenguaje DOT). Su función es crear el string necesario para dibujar el árbol jerárquico que puedes ver en la pestaña de reportes AST.

```
Backend > Util > JS GeneradorAST.js > [?] generarDOT
5  const generarDOT = (ast) => {
6
7  };
8
9  const recorrer = (nodo, padreId) => {
10    if (nodo === null || nodo === undefined) return;
11
12    // Si es un arreglo (lista de instrucciones)
13    if (Array.isArray(nodo)) {
14      nodo.forEach(item => recorrer(item, padreId));
15      return;
16    }
17
18    if (typeof nodo === 'object') {
19      let actualId = `n${contador++}`;
20
21      // Limpiamos el nombre del tipo para evitar errores de sintaxis en DOT
22      let tipo = nodo.tipo ? nodo.tipo.toString().replace(/"/g, '\\"') : "nodo";
23
24      cuerpoDot += ` ${actualId} [label="${tipo}"]; \n`;
25      cuerpoDot += ` ${padreId} -> ${actualId}; \n`;
26
27      // Recorremos los hijos
28      for (let propiedad in nodo) {
29        if (propiedad !== 'tipo') {
30          const valor = nodo[propiedad];
31
32          if (Array.isArray(valor) || (valor !== null && typeof valor === 'object')) {
33            recorrer(valor, actualId);
34          } else if (valor !== null && valor !== undefined) {
35            // Es un valor terminal (ID, número, string)
36            let terminalId = `t${contador++}`;
37            let labelTerminal = valor.toString().replace(/"/g, '\\"');
38            cuerpoDot += ` ${terminalId} [label="${labelTerminal}", shape=ellipse, fillcolor="#fff9c4"]; \n`;
39            cuerpoDot += ` ${actualId} -> ${terminalId}; \n`;
40          }
41        }
42      }
43    }
44  };
45
46  };
47
48  };
49
50  };
51
52  };
53
54  };
55
56  };
57
58  };
```

Interprete.js:

El intérprete procesa el AST generado por Jison mediante un patrón de diseño de recorrido recursivo. Su objetivo es transformar una estructura de datos estática en un programa dinámico. El proceso se divide en las siguientes etapas:

```
15  /* — Salida acumulada de fmt.Println / fmt.Print — */
16  let salida = [];
17
18  /* =====
19  HELPERS
20  ===== */
21  const registrar = (id, tipoSimbolo, tipoDato, ambito) => {
22    listaSimbolos.push(new Simbolo(id, tipoSimbolo, tipoDato, ambito, 0, 0));
23  };
24
25  const valorRune = (r) => {
26    if (typeof r === 'number') return r;
27    if (typeof r === 'string' && r.startsWith('"') && r.endsWith('"')) {
28      const inner = r.slice(1, -1);
29      if (inner === '\\n') return 10;
30      if (inner === '\\t') return 9;
31      if (inner === '\\r') return 13;
32      if (inner.startsWith('\\')) return inner.charCodeAt(1);
33      return inner.charCodeAt(0);
34    }
35    return 0;
36  };
37
38  const esRune = (v) =>
39    typeof v === 'string' && v.startsWith('"') && v.endsWith('"');
40
41  const resolverRune = (v) => esRune(v) ? valorRune(v) : v;
42
43  const tipoDe = (v) => {
44    if (v === null || v === undefined) return 'nil';
45    if (typeof v === 'boolean') return 'bool';
46    if (Number.isInteger(v)) return 'int';
47    if (typeof v === 'number') return 'float64';
```